Cheatsheet Backend Go

Reglas de oro (no se rompen)

- **Bases de dev/staging son compartidas.** Nada de `POST/PUT/DELETE` ni migraciones contra ellas sin que el equipo lo pida.
- **Producción jamás** desde tu máquina: solo por el pipeline.
- Migraciones automáticas al arrancar **apagadas** cuando apuntas a una base compartida.
- `config.json`, `certs/`, `.env`, binarios y `docs/` (swag) **nunca** en commits. Verifica: `git check-ignore -v <archivo>`.
- No apagues la autenticación "para probar rápido": se te olvida prenderla.
- Si tu servicio **consume** una cola compartida, no lo conectes desde local: le robas mensajes al entorno.
- `example:` de Swagger nunca con valores reales.

Mapa de un backend de microservicios

Entorno	Para qué
local	tu máquina; bases propias en Docker, rompe lo que quieras
dev / staging	probar ramas y QA; bases **compartidas**
production	clientes reales; solo vía pipeline

Recurso	Desde tu máquina	Dentro de docker
Postgres	`<host público>:<puerto publicado>`	`dbpostgres:5432`
Rabbit	`<host>:<puerto publicado>`	`rabbit:5672`
HTTP	`https://<svc>.<dominio>`	`http://<servicio>`

Los alias de la red docker **no resuelven** fuera del servidor: en local cambia alias por host público.

Repos típicos

- `servicio-<dominio>`: un microservicio por dominio (auth, pedidos, inventarios…).
- `libs/platform`: librería compartida privada (errores, config, auth, mensajería).
- `infra/deploy`: compose, configs por entorno, pipelines.

Levantar un servicio Go ajeno: checklist

#	Paso	Comando clave / éxito
1	Acceso	`go env -w GOPRIVATE=github.com/tu-org/*` · `git ls-remote https://github.com/tu-org/libs.git`
2	Dependencias	`go mod download` · `go install github.com/swaggo/swag/cmd/swag@<ver go.mod>` · `swag init -g <main>.go` · `go build ./...`
3	Config	`cp config.example.json config.json`; alias docker -> hosts reales; migraciones automáticas off; `python3 -m json.tool config.json`
4	Certificados	si la base exige TLS: `ca.crt/client.crt/client.key` fuera de git, `chmod 600` a la llave
5	Red	`nc -vz -w 5 <host> <puerto>` a cada dependencia
6	Conexión	`go run <main>.go migrate status` (solo lee)
7	Arranque	`go run <main>.go` -> `Listening and serving HTTP on :<puerto>` · `curl` a una ruta pública

```
# inspeccionar certificados (LibreSSL en macOS no tiene -ext)
openssl x509 -in client.crt -noout -subject
openssl x509 -in ca.crt -noout -text | grep -A1 "Subject Alternative Name"
# cargar variables de un .env en la terminal actual
set -a; source .env; set +a
```

Terminal y diagnóstico

```
pwd; ls -la; cd -  # dónde estoy / qué hay / regresar
grep -rn "txOrDB" src/  # buscar en código
find . -name "*_test.go" | wc -l
tail -f app.log | grep -i error
lsof -i :1099  # quién usa el puerto
kill <PID>
nc -vz -w 5 host 5432  # ¿hay red?
curl -s -i localhost:1099/salud # -i muestra código y headers
curl -s -X POST -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" -d '{"a":1}' URL
export VAR=valor; echo $VAR  # solo vive en ESTA terminal
chmod 600 archivo; ls -l archivo
python3 -m json.tool config.json  # validar JSON
```

Git y convenciones del equipo

```
git switch -c feature/123-alta-cajas develop
git status; git diff; git diff --staged
git add -p  # preparar por partes
git commit -m "feat(cajas): alta de cajas (#123)"
git fetch origin && git rebase origin/develop
git stash / git stash pop
git log --oneline --graph -15
git restore archivo.go  # descartar cambios locales
git check-ignore -v config.json
```

Conventional Commits: `feat` `fix` `refactor` `perf` `docs` `test` `chore` · Ramas `feature/<ticket>-slug` desde `develop` (o `main`) · PR pequeño.

Git y convenciones del equipo

> Lo que entra a git se queda en el historial: borrar un secreto en el siguiente commit **no** lo arregla; hay que rotarlo.

Go: herramienta

```
go build ./...  # compilar todo
go vet ./...  # en cero
gofmt -l .  # sin salida = formateado
go test ./... -v -race
go test ./pkg/ -run TestX -v  # un test
go test -tags reto_caja ./...  # incluye //go:build reto_caja
go test -cover ./...
go mod download  # tidy borra docs/ si no existe: swag primero
go env GOPRIVATE GOMODCACHE
go doc fmt.Errorf
```

Módulo = `go.mod` · paquete = carpeta · **Mayúscula** = exportado.

Go: lo básico que más se usa

```
x := 10; var s string; const Max = 3
// valor cero: 0, "", false, nil (map nil: leer ok, escribir PANIC)
xs := []int{1, 2}; xs = append(xs, 3)  // siempre reasignar
vacio := make([]T, 0, n)  // JSON: [] y no null
m := map[string]int{}; v, ok := m["k"]; delete(m, "k")
for i, v := range xs { }  for k, v := range m { }
switch tipo { case "A", "B": ... default: ... }

type Almacen struct { ID uint; Clave string }
func (a Almacen) Etiqueta() string { ... }  // valor: copia
func (a *Almacen) Activar()  { ... }  // puntero: modifica
func NewX(dep Dep) *X { return &X{dep: dep} }

s = strings.TrimSpace(strings.ToUpper(s))
n, err := strconv.Atoi("12"); id, err :=
strconv.ParseUint(p, 10, 64)
fmt.Sprintf("%s %d %v %+v %q", ...)
defer f.Close()  // al salir de la función
```

Go: errores

```
var ErrNoExiste = errors.New("no existe")  // centinela
return fmt.Errorf("almacén %d: %w", id, err)  // %w envuelve
if errors.Is(err, ErrNoExiste) { }  // busca VALOR
var e *MiError; if errors.As(err, &e) { e.Codigo } // busca TIPO
func (e *MiError) Unwrap() error { return e.Causa }
```

- Revisa `err` justo después de la llamada. Nunca compares por texto.
- Entre capas, envuelve con contexto (`%w` o la librería de errores del equipo), no `return err` crudo.

Cheatsheet Backend Go

Go: interfaces, context, concurrencia, genéricos

```
type AlmacenReader interface {  // PUERTO chico
  PorID(ctx context.Context, id uint) (*models.Almacen,
error)
}
var _ AlmacenReader = (*AlmacenRepositoryImpl)(nil) //
chequeo en compilación

ctx, cancel := context.WithTimeout(ctx, 3*time.Second)
defer cancel()  // SIEMPRE
select { case r := <-ch: ...; case <-ctx.Done(): return
ctx.Err() }
// en gin: ctx.Request.Context()

ch := make(chan T, 1)  // buffer 1: la goroutine no se queda
colgada
var wg sync.WaitGroup; wg.Add(1); go func(){ defer wg.Done()
}(); wg.Wait()
var mu sync.Mutex; mu.Lock(); defer mu.Unlock()

func Mapear[T, U any](xs []T, f func(T) U) []U { ... }
```

- Gin atiende cada petición en su goroutine: estado compartido con `Mutex`. Detecta carreras con `-race`.
- En servicios, envuelve `go func(){}` en un helper que **recupere pánicos**, propague el `ctx` (TraceID) y tenga timeout.

Go: tests (estilo del equipo)

```
func TestRegistrar(t *testing.T) {
  casos := []struct{ nombre string; entra models.X; codigo
int }{
    {"válido", models.X{...}, 0},
    {"sin nombre", models.X{}, http.StatusBadRequest},
  }
  for _, c := range casos {
    t.Run(c.nombre, func(t *testing.T) {
      repo := nuevoFalso()  // fake en memoria
      svc := NewXService(repo, repo, &uowFalso{})
      err := svc.Registrar(context.Background(), &c.entra)
      if c.codigo != 0 && !apperr.IsCode(err, c.codigo) {
t.Fatalf(...) }
    })
  }
}
// HTTP sin abrir puertos
req := httptest.NewRequest(http.MethodGet, "/x", nil)
rec := httptest.NewRecorder(); router.ServeHTTP(rec, req) //
rec.Code, rec.Body
```

- `t.Errorf` sigue · `t.Fatalf` corta · `t.Helper()` en utilerías.
- Fake = implementa el puerto y **registra** qué se pidió (`escrituras []string`).
- Tests de integración que hacen `Skip` sin `TEST_DSN`: un `ok` todo en Skip no verificó nada. Usa `-v`.

HTTP: códigos

200/201	OK / creado	**401**	sin token o token inválido
400	datos inválidos	**403**	sin permiso (rol/política)
404	no existe	**409**	duplicado / estado inválido
500	error nuestro	**503**	dependencia caída

GET consulta · POST crea · PUT/PATCH modifica (el borrado lógico suele ser PUT) · DELETE borra.

Gin

```
r := gin.New(); r.Use(gin.Logger(), gin.Recovery())
g := r.Group("almacenes")
g.GET("", ctrl.GetAll)  // c.Query("activos")
g.GET("/:id", ctrl.GetByID)  // c.Param("id")
g.POST("", ctrl.Alta)

type AltaDTO struct {
  Clave string `json:"clave" binding:"required"
example:"ALM-01"`
}
var req AltaDTO
if err := c.ShouldBindJSON(&req); err != nil { /* 400 */ }
c.JSON(http.StatusOK, gin.H{"k": v})

func Mw() gin.HandlerFunc { return func(c *gin.Context) {
  if malo { c.AbortWithStatusJSON(401, ...); return }
  c.Next() } }
```

Orden load-bearing: `r.Use(mw)` solo afecta rutas registradas **después**. Patrón: swagger y rutas públicas -> `Use(auth)` -> rutas protegidas. Todo lo registrado antes del middleware = público. Congela el inventario de rutas con un test.

Handler: bind -> mapper DTO a modelo -> service -> `general.Responder`. Sin reglas de negocio ni SQL. `binding:"required"` acepta `" "`: recortar va en el service.

Swagger (swaggo)

```
// Alta godoc
// @Summary  Registra un almacén
// @Tags  almacenes
// @Accept  json
// @Produce  json
// @Security  BearerToken
// @Param  almacen body dtos.AlmacenAltaDTO true "Almacén"
// @Param  id path int true "ID"  |  activos query bool
false "Solo activos"
// @Success  201 {object} dtos.RespuestaAlmacen
// @Failure  400 {object} dtos.RespuestaError
// @Router  /almacenes [post]
```

- `docs/` es generado y **no se versiona**; el main lo importa (`_ ".../docs"`). Clon limpio: `no required module provides package .../docs` -> `swag init -g <main>.go`.
- `swag` en la **misma versión** que `go.mod` (si no: `unknown field LeftDelim`).
- Regenera tras tocar handlers/DTOs. UI: `/swagger/index.html`.

GORM v2: trampas

Haces	Pasa	Haz
`Find` y esperas `ErrRecordNotFound`	nunca llega; 404 muerto	`First`, o `RowsAffected == 0`
`Updates(struct)`	ignora `false`, `0`, `""`	`Updates(map[string]any{...})` o `Select`
`"id = " + id`	inyección SQL	`Where("id = ?", id)`
`Rollback()` en el `*gorm.DB` raíz	*invalid transaction* en todo el servicio hasta reiniciar	solo `uow.Do`
Deducir tipo de columna por el struct	el esquema viene de migraciones	lee el `.sql`

```
txOrDB(ctx, r.db).Where("id = ?", id).First(&x)
txOrDB(ctx, r.db).Preload("Valores").Where(...).Find(&lista)
txOrDB(ctx, r.db).Create(&x)
txOrDB(ctx, r.db).Model(&X{}).Where("id = ?",
id).Updates(map[string]any{"activo": false})
txOrDB(ctx, r.db).Model(&X{}).Where(...).Count(&total)
```

UnitOfWork + txOrDB

```
err := s.uow.Do(ctx, func(ctx context.Context) error {
  if err := s.writer.Insertar(ctx, a); err != nil { return
err }
  return s.otro.Insertar(ctx, b)  // misma transacción (va
en ctx)
})
```

- `Do` **commitea** si `fn` devuelve `nil`: un `return nil` temprano persiste lo hecho. Para abortar sin error al cliente: centinela + `IsCode` fuera del `Do`.
- `Do` anidado = transacción **independiente**, no savepoint. Abre una sola vez arriba.
- Los adaptadores siempre usan `txOrDB(ctx, r.db)`.

Migraciones y Postgres

```
go run <main>.go migrate status  # permitido: solo lee
go run <main>.go migrate up  # SOLO en tu base local /
práctica
migrate create -ext sql -dir migrations -seq <nombre> #
golang-migrate
```

- SQL embebido (`//go:embed`): un `.sql` nuevo no existe hasta recompilar.
- Escribe SQL para la versión de Postgres del **servidor**, no la de tu laptop (p. ej. `CREATE OR REPLACE TRIGGER` requiere PG 14+).
- Una migración aplicada **no se edita**: se crea otra. FKs con `REFERENCES` + índice.
- Bases expuestas suelen exigir TLS (`pg_hba` con `hostssl`): `sslmode=verify-full` + certificados.

```
psql "host=127.0.0.1 port=5433 user=dojo dbname=dojo"
\dt  \d tabla  \x  \q
SELECT ... FROM a LEFT JOIN b ON b.a_id = a.id WHERE ...
GROUP BY ... ORDER BY ... LIMIT 20;
```

Docker y Compose

```
docker ps -a; docker images; docker logs -f --tail 100 <c>
docker exec -it <c> sh
docker build -t nombre:tag .
docker compose up -d [servicio]  # --build para reconstruir
docker compose ps / logs -f api / restart api
docker compose exec dbpostgres psql -U <u>
docker compose down  # conserva volúmenes
docker compose down -v  # BORRA volúmenes (solo práctica)
```

- `ports: "127.0.0.1:5433:5432"` = **HOST:CONTENEDOR**; `127.0.0.1` no expone a la red.
- Contenedor a contenedor: **alias + puerto interno**. Tu máquina: **puerto publicado**.
- Multi-stage: `golang` compila, `debian:12-slim/distroless` ejecuta; la imagen **no trae** `config.json` (se monta).
- `failed to connect to the docker API` = Docker no está corriendo.
- Contenedor reinicia en bucle: `docker logs --tail 100` y busca el `Fatal` (config, Postgres, red).

Arquitectura hexagonal

Carpeta	Qué va
`domain/models`	Entidades
`domain/repositories`	Puertos `…Reader`/`…Writer`, `UnitOfWork`
`domain/services`	Interfaz + `…Impl`: lógica de negocio
`infrastructure/persistence`	Adaptadores GORM, `txOrDB`
`infrastructure/web/…`	`controllers` `dtos` `mappers` `routes`
`infrastructure/external`	Otros servicios, Auth0, casbin, APIs
`infrastructure/messaging`	Rabbit, `LISTEN`, websocket
`infrastructure/app/app.go`	**Composition root**: único lugar que cablea

Dominio **no importa** gin, gorm ni `infrastructure`. Lo vigila `src/architecture_test.go`.

Receta: operación nueva

1 puerto (si hace falta) · 2 service (interfaz + Impl, valida, `uow.Do` si >1 escritura) · 3 test con fake · 4 adaptador(es) · 5 DTO -> mapper -> handler + swagger -> ruta **después** del middleware · 6 cableado en `app.go` · 7 `swag init`, build, vet, fmt, test, `curl`.

Nombres

Rol en inglés (`Reader`, `Service`, `Impl`), negocio en español (`Almacen`, `PorID`, `Insertar`). Los nombres en español **no se renombran** (contrato JSON, rutas, columnas). Un concepto = **mismo nombre de archivo** en todas las capas.

Errores estilo librería interna (`pkg/apperr`)

```
return nil, apperr.New(http.StatusNotFound, "El almacén no existe", err)
return nil, apperr.Wrap(err)  // entre capas
if apperr.IsCode(err, http.StatusNotFound) { } // decidir
apperr.Response(err, ctx)  // error del service
apperr.CtrlResponse(400, msg, err, ctx)  // error del borde HTTP
log.Printf("código %d: %v", apperr.Code(err), err)
```

4 trampas típicas (el dojo las trae a propósito):
1. `AppError` **no tiene** `Unwrap()`: `errors.Is` no ve la causa -> usa `IsCode`.
2. `apperr.Wrap(nil)` **no es nil**: si absorbes not-found, `return nil, nil` explícito.
3. `apperr.New(code, msg, nil)` **revienta al responder**: siempre causa (`errors.New(msg)`).
4. Nunca `return err` crudo entre paquetes.

Librería compartida: config, auth, ecosistema

- **Módulo privado:** `go env -w GOPRIVATE=github.com/tu-org/*` + credencial git (SSH o token). Versiona con **semver** y tags `vX.Y.Z`.
- Probar cambios sin publicar: `replace github.com/tu-org/libs => ../libs` en `go.mod` (**nunca** en commit) o `go work`.
- **config:** si la librería cae en una config de prueba cuando no encuentra el archivo, **no falla**: lee el log de arranque si "no ves tus datos".
- **auth:** JWT + autorización por rol (p. ej. casbin). **Ruta nueva = 403** hasta darle permiso en **cada entorno**.
- **Token servicio a servicio** (`X-App-Token`): solo entre tus servicios, **nunca** hacia terceros. Cliente HTTP siempre con timeout.
- Parsear `Authorization` con `strings.Cut`, no `Split(...)[1]` (panic = 500 en vez de 401).
- **Trace:** TraceID viaja en el `ctx`; pásalo siempre.
- **Shutdown:** cada tarea en segundo plano respeta `ctx.Done()`.
- Leer el código de una dependencia: `$(go env GOMODCACHE)/<módulo>@<versión>/` o `go doc`.

Seguridad y secretos

Secreto	Vive en
Contraseñas BD/Rabbit	`config.json` (ignorado) · gestor de secretos / variables del pipeline
Certificados	`certs/<entorno>/` (ignorado)
`JWT_SECRET`, `DOJO_TOKEN`	`.env` del servidor (ignorado)
Llaves privadas, API keys	variables de entorno / vault

- SQL siempre con `?` · timeouts en todo I/O externo · `subtle.ConstantTimeCompare` para tokens · no loguear headers `Authorization` ni configs completos.
- Ruta expuesta = registrada antes del middleware de auth. Congela el inventario de rutas públicas con un test.

Seguridad y secretos

- Nodo nuevo en config -> también en `config.example.json` con placeholder.

Proceso: del ticket a producción

Ticket -> rama `feature/<ticket>-slug` -> código + tests + swagger + migración -> verificación local -> PR (CI: build, vet, test, lint) -> revisión + QA en **staging** -> merge -> pipeline despliega a **production**.

```
# .github/workflows/ci.yml (esqueleto)
on: [pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with: { go-version-file: go.mod }
      - run: go vet ./... && go test -race ./...
```

Checklist antes de desplegar

- Ruta nueva -> permisos/roles en cada entorno.
- Llave nueva de config -> `config.example.json` + config de cada entorno **antes** del código.
- Migración -> quién y cuándo se aplica en producción: coordínalo.

PR que se revisa rápido

Título `feat(cajas): alta de cajas (#123)` · qué y por qué · **cómo se probó** (comandos + `curl`; si no pudiste ejercitarlo, dilo) · riesgos · pasos de despliegue. Verificación: `go build ./...` · `go vet ./...` · `gofmt -l .` · `go test ./...`.

Cheatsheet Backend Go

Go · Gin · Swagger · Postgres/GORM · Docker · Hexagonal · Librerías compartidas · CI/CD · Dojo

Error -> solución

Mensaje	Arreglo
`410 Gone` / `proxy.golang.org …tu-org/libs`	`GOPRIVATE`
`Node.js environment variables are disabled…`	VS Code: off `git.terminalAuthentication` o `GIT_ASKPASS=`
`terminal prompts disabled`	`git ls-remote` una vez para guardar credencial
`no required module provides package …/docs`	`swag init -g <main>.go`
`swag: command not found`	`~/go/bin` al `PATH`
`unknown field LeftDelim`	`swag` en versión de `go.mod`
`lookup dbpostgres: no such host`	alias docker -> host público
`no pg_hba.conf entry … SSL off`	`sslmode` + certs
`private key file has group or world access`	`chmod 600 client.key`
`certificate is valid for X, not Y`	host del SAN o `verify-ca`
`certificate signed by unknown authority`	`ca.crt` del entorno correcto
`certificate authentication failed for user`	CN de `client.crt` = usuario
`i/o timeout`	`nc -vz`; si falla, no es tu código
`bind: address already in use`	`lsof -i :<puerto>`
403 en ruta nueva	permiso/rol en ese entorno
"usando config por defecto" en el log	ruta de `CONFIG` mal
`invalid transaction` en todo	alguien hizo `Rollback()` en la raíz: reiniciar y corregir
`concurrent map writes`	`Mutex`; corre `-race`

Dojo (Claude Code en la raíz del repo)

`/progreso`	bitácora y qué sigue
`/pista`	pista de 3 niveles, sin solución
`/revisar-reto`	tests + revisión estilo PR
`/examen`	autoevaluación del módulo

```
# katas (practica/katas)
go test ./basico/ -v
go test ./intermedio/ -race -v
# dojo-api (practica/dojo-api) · :1099 · token dojo-local-inseguro
swag init -g main.go && go test ./... && go run main.go
go test -tags reto_seguridad ./src/infrastructure/app/ -run Reto -v  # A · mód 04
go test -tags reto_desactivar ./... -run Reto -v  # B · mód 08
go test -tags reto_caja ./... -run Reto -v  # C · mód 12
docker compose up -d dbpostgres  # 127.0.0.1:5433 / dbpostgres:5432
go run main.go migrate status|up
```

Dojo (Claude Code en la raíz del repo)

Cinturones: blanco 00–01 · amarillo 02–03 · naranja 04–05 · verde 06–07 · azul 08–09 · café 10–11 · negro 12.

Antes de decir "ya quedó"

`swag init -g <main>.go` (si tocaste handlers/DTOs)
`go build ./...` · `go vet ./...` · `gofmt -l .` (vacío)
`go test ./... -race` (lee con `-v` si hay Skip)
`curl` al endpoint (local, sin escribir en bases compartidas)
`git status`: sin `config.json`, `certs/`, `.env`, `docs/`, binarios
Rutas nuevas después del middleware · permisos anotados para el despliegue
Lista vacía sale `[]`, no `null`
Errores: sin `return err` crudo, sin `Wrap(nil)`, sin `New(…, nil)`